JAVASCRIPT CLASS TEST

Section A: Multiple-Choice Questions

Q1. What will be the output?

- A. null, number, array
- B. object, number, object
- C. object, NaN, array
- D. undefined, number, object

```
console.log(typeof null);
console.log(typeof NaN);
console.log(typeof []);
```

Q2. Predict the output.

- A. 205, 200, 24
- B. 205, 200, NaN
- C. NaN, 200, NaN
- D. 2203, 403203, 24

```
console.log(2 + "203");
console.log(403 - "203");
console.log(2 * "12abc");
```

Q3. Which statement about let, const, and var is correct?

- A. var is block-scoped.
- B. let can be redeclared in the same scope without error.
- C. A const object cannot have its properties modified.
- D. let and const are block-scoped, while var is function-scoped.

Q4. What is the output?

- A. Pass, Eligible
- B. Pass, followed by a ReferenceError
- C. ReferenceError, ReferenceError
- D. undefined, undefined

```
let score = 12;

if (score > 10) {
    var result = "Pass";
    let status = "Eligible";
}

console.log(result);
console.log(status);
```

Q5. Which loop is most suitable when you need to iterate over the values of an array?

- A. for...in
- B. for...of
- C. if...else
- D. switch

Q6. What is the main difference between return and console.log() inside a function?

- A. Both return values to the caller.
- B. console.log() returns the calculated result.
- C. return sends a value back to the caller; console.log() displays a value.
- D. return can only be used in arrow functions.

Q7. What does the rest parameter do in this function?

- A. Converts all arguments into a string.
- B. Collects the arguments into an array.
- C. Expands an existing array into arguments.
- D. Allows only one argument.

```
function calculateTotal(...prices) {
    return prices.length;
}
```

Q8. Which statement about arrow functions and “this” is correct?

- A. Arrow functions always bind this to the object that calls them.
- B. Regular functions always bind this to the object in which they are defined.
- C. Arrow functions inherit this from their surrounding lexical context.
- D. Arrow functions cannot access object properties.

Section B: Predict the Output

Q9. Operators and type coercion

Predict the output of each statement and explain why the results differ.

```
console.log("5" + 2);
console.log("5" - 2);
console.log(true + 2);
console.log(Boolean("false"));
```

Q10. Scope and variable declarations

Write the output in order. Explain why amount and discount behave differently.

```
var amount = 100;

function calculate() {
    var amount = 200;
    let discount = 20;

    if (true) {
        var amount = 300;
        let discount = 50;
        console.log(amount, discount);
    }

    console.log(amount, discount);
}

calculate();
console.log(amount);
```

Q11. Functions and return values

What will be printed? Rewrite the function so that result stores the calculated price.

```
function calculatePrice(price, quantity) {
    console.log(price * quantity);
}

const result = calculatePrice(100, 3);
console.log(result);
```

Q12. Array mutation

Predict the output. Why are modifications to the array allowed even though it was declared using const?

```
const products = ["Milk", "Bread", "Rice"];

products[1] = "Eggs";
products.push("Oil");

console.log(products);
console.log(products.length);
```

Q13. Object methods and this

Predict the output. What does this refer to during this method call?

```
const employee = {
    name: "Aman",
    salary: 30000,

    showDetails() {
        console.log(this.name, this.salary);
    }
};

employee.showDetails();
```

Q14. Rest parameters and spread syntax

Predict both outputs and explain the difference between the two function calls.

```
function addNumbers(...numbers) {
    return numbers.length;
}

const values = [10, 20, 30, 40];

console.log(addNumbers(values));
console.log(addNumbers(...values));
```

Section C: Debug the Code

Q15. Fix the variable scope issue

The developer wants to print the message after the if block, but the code produces an error. Identify the error, correct the code, and explain why the original code fails.

```
if (true) {
    let message = "Order placed successfully";
}
```

```
console.log(message);
```

Q16. Fix the shopping cart calculation

Identify the logical error, correct the loop, and explain why the original code produces an incorrect result.

```
function calculateCartTotal(...prices) {
    let total = 0;

    for (let i = 0; i <= prices.length; i++) {
        total += prices[i];
    }

    return total;
}

console.log(calculateCartTotal(100, 200, 300));
```

Q17. Fix the object method

Explain why the expected username may not be printed. Rewrite the method correctly and state one situation where using an arrow function is appropriate.

```
const user = {
    userName: "Rahul",

    showName: () => {
        console.log(this.userName);
    }
};

user.showName();
```

Q .18 Type Coercion — Predict the Output

console.log([] + []);

console.log([] + {});

console.log("5" - - "2");

console.log(null + 1);

Predict the output of each statement and explain why.

Q19. Scope and Hoisting

console.log(a);

var a = 10;

console.log(b);

let b = 20;

- Which statement executes successfully?
- Which statement produces an error?
- Explain the role of hoisting and the Temporal Dead Zone (TDZ).

JavaScript Interview-Level Questions

1. What is the difference between == and ===?
2. What is the difference between `undefined` and `null`?
3. What is the difference between `for...in` and `for...of`?
4. Can a function return another function? Give a real-world use case.
5. What happens if a function has no explicit `return` statement?
6. What is the difference between passing a primitive value and passing an object to a function?
7. What is the difference between shallow copying and deep copying an object?
8. Why is `NaN !== NaN` true?
9. What is the difference between function declarations and function expressions regarding hoisting?
10. When would you use an arrow function instead of a regular function?